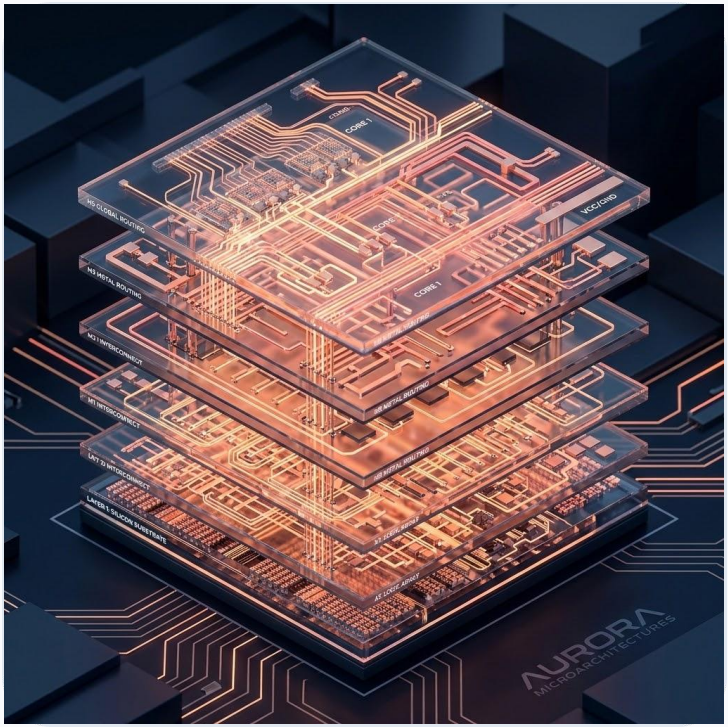
Máquina Multinivel

y Lógica Digital

Fundamentos de

Arquitectura de Computadoras



Recursos de una computadora

CPU - Procesador

Encargado de ejecutar las instrucciones y procesar los datos.

Memoria Principal - RAM

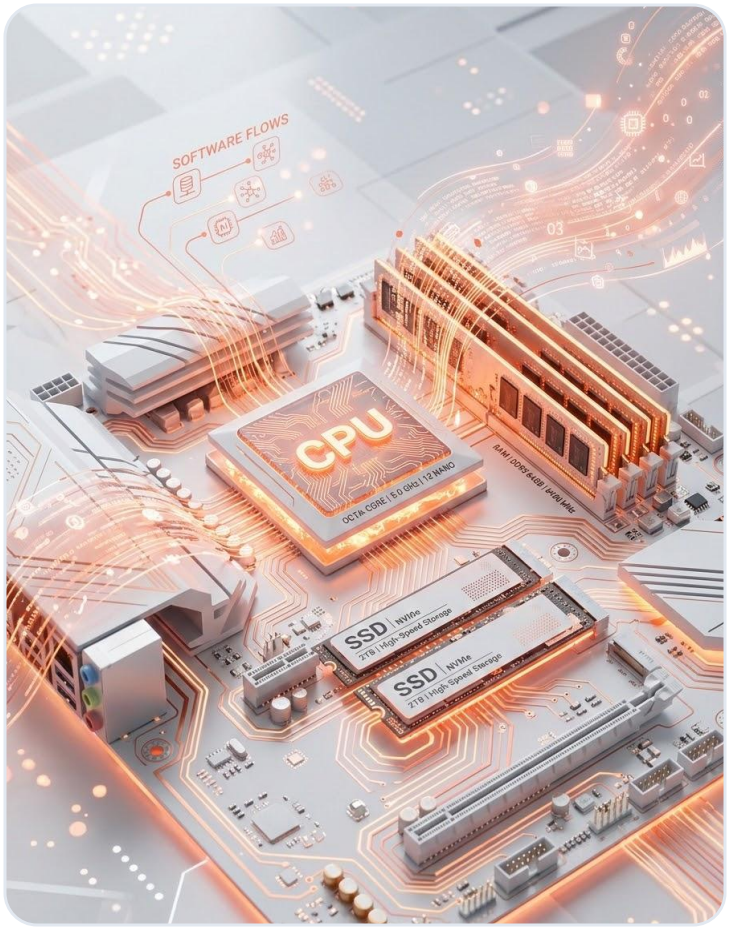
Almacenamiento de acceso rápido y temporal para los programas en ejecución.

Dispositivos de Entrada/Salida

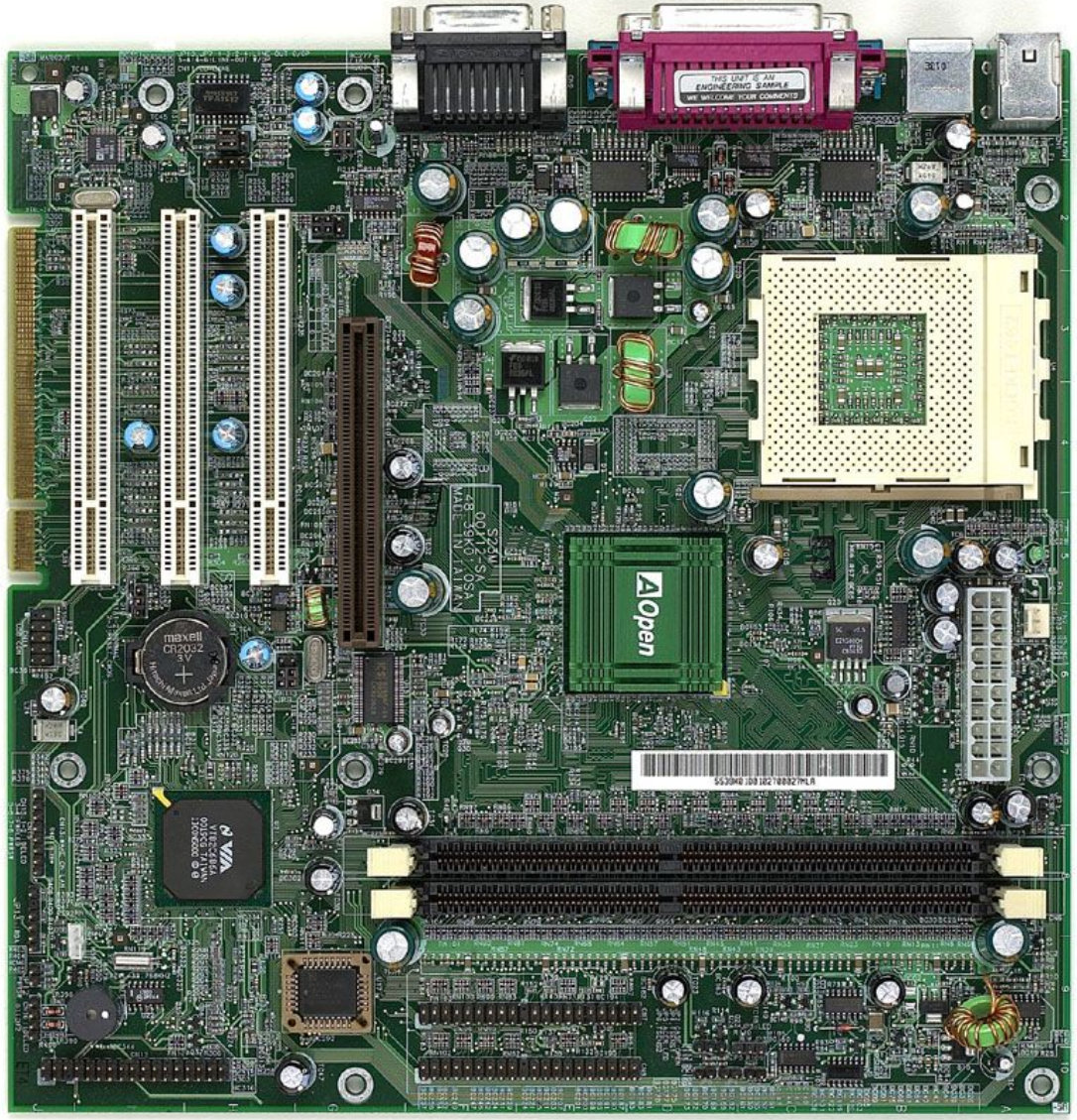
Periféricos de interacción y almacenamiento secundario permanente para preservar la información.

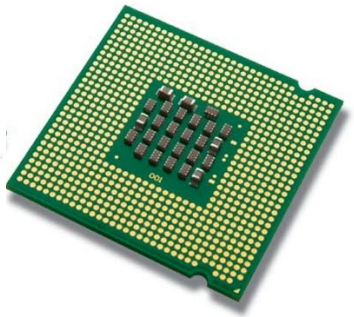
Información (Software)

El conjunto de programas y datos.



Placa Madre (Motherboard)





Máquina multinivel

Máquina multinivel

Programa en diferentes niveles

LENGUAJE DE ALTO NIVEL

```
int m[256];
int main() {
    int* i = &m[0];
    int c = 3;
    *(i+1) = 1;
    do {
        *(i+1) *= c;
        c--;
    } while (c != 0);
    printf("%d\n", *(i+1));
}
```

SE COMPILA

COMPILADOR

LENGUAJE MÁQUINA

```
          00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
00000000  50 10 D0 B0 00 01 00 04 D0 90 00 03 CC 74 CC 00
00000010  04 D0 91 FF FF CC 85 00 0D 90 00 01 A4 90 04 01
00000020  CC 91 00 04 D0 80 00 02 0F +
```

SE INTERPRETA

Programa en diferentes niveles

LENGUAJE DE ALTO NIVEL

```
int m[256];
int main() {
    int* i = &m[0];
    int c = 3;
    *(i+1) = 1;
    do {
        *(i+1) *= c;
        c--;
    } while (c != 0);
    printf("%d\n", *(i+1));
}
```

SE COMPILA

LENGUAJE ASSEMBLER

```
        mov   edx, ds
        mov   [edx+4], 1
        mov   cx, 3
otro:   mul   [edx+4], cx
        add   cx, -1
        jnz   otro
fin:    mov   al, 1
        mov   cx, 0x0401
        add   edx, 4
        sys   0x2
        stop
```

SE TRADUCE

LENGUAJE MÁQUINA

```
          00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
00000000  50 10 D0 B0 00 01 00 04 D0 90 00 03 CC 74 CC 00
00000010  04 D0 91 FF FF CC 85 00 0D 90 00 01 A4 90 04 01
00000020  CC 91 00 04 D0 80 00 02 0F +
```

SE INTERPRETA

Programa en diferentes niveles

LENGUAJE DE ALTO NIVEL

```
int m[256];
int main() {
    int* i = &m[0];
    int c = 3;
    *(i+1) = 1;
    do {
        *(i+1) *= c;
        c--;
    } while (c != 0);
    printf("%d\n", *(i+1));
}
```

LENGUAJE ASSEMBLER

```
        mov   edx, ds
        mov   [edx+4], 1
        mov   cx, 3
otro:   mul   [edx+4], cx
        add   cx, -1
        jnz   otro
fin:    mov   al, 1
        mov   cx, 0x0401
        add   edx, 4
        sys   0x2
        stop
```

LENGUAJE MÁQUINA

```
          00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
00000000  50 10 D0 B0 00 01 00 04 D0 90 00 03 CC 74 CC 00
00000010  04 D0 91 FF FF CC 85 00 0D 90 00 01 A4 90 04 01
00000020  CC 91 00 04 D0 80 00 02 0F +
```

Programa en diferentes niveles

LENGUAJE DE ALTO NIVEL

```
int m[256];
int main() {
    int* i = &m[0];
    int c = 3;
    *(i+1) = 1;
    do {
        *(i+1) *= c;
        c--;
    } while (c != 0);
    printf("%d\n", *(i+1));
}
```

LENGUAJE ASSEMBLER

```
        mov   edx, ds
        mov   [edx+4], 1
        mov   cx, 3
otro:   mul   [edx+4], cx
        add   cx, -1
        jnz   otro
fin:    mov   al, 1
        mov   cx, 0x0401
        add   edx, 4
        sys   0x2
        stop
```

LENGUAJE MÁQUINA

```
          00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
00000000  50 10 D0 B0 00 01 00 04 D0 90 00 03 CC 74 CC 00
00000010  04 D0 91 FF FF CC 85 00 0D 90 00 01 A4 90 04 01
00000020  CC 91 00 04 D0 80 00 02 0F +
```

Programa en diferentes niveles

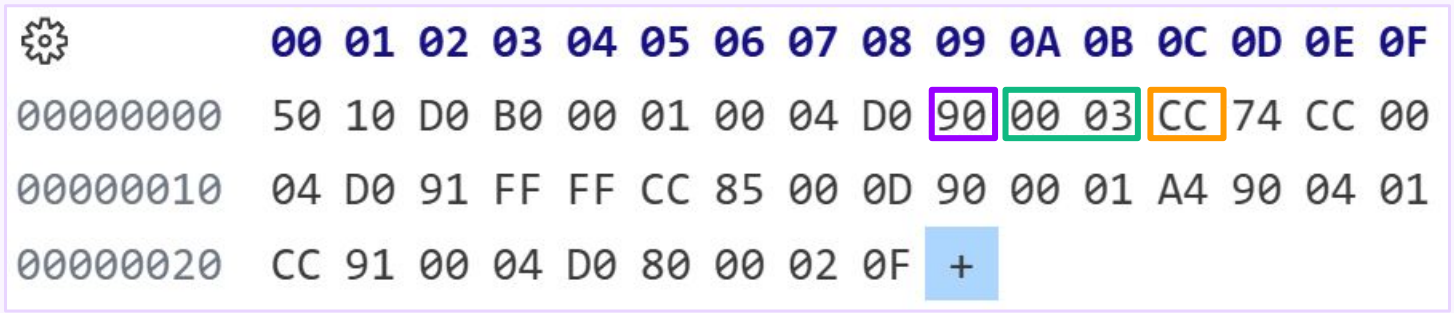
LENGUAJE DE ALTO NIVEL

```
int m[256];
int main() {
    int* i = &m[0];
    int c = 3;
    *(i+1) = 1;
    do {
        *(i+1) *= c;
        c--;
    } while (c != 0);
    printf("%d\n", *(i+1));
}
```

LENGUAJE ASSEMBLER

```
        mov   edx, ds
        mov   [edx+4], 1
        mov   cx, 3
otro:   mul   [edx+4], cx
        add   cx, -1
        jnz   otro
fin:    mov   al, 1
        mov   cx, 0x0401
        add   edx, 4
        sys   0x2
        stop
```

LENGUAJE MÁQUINA



Memoria

Celda

Byte

Dirección

Direccionamiento

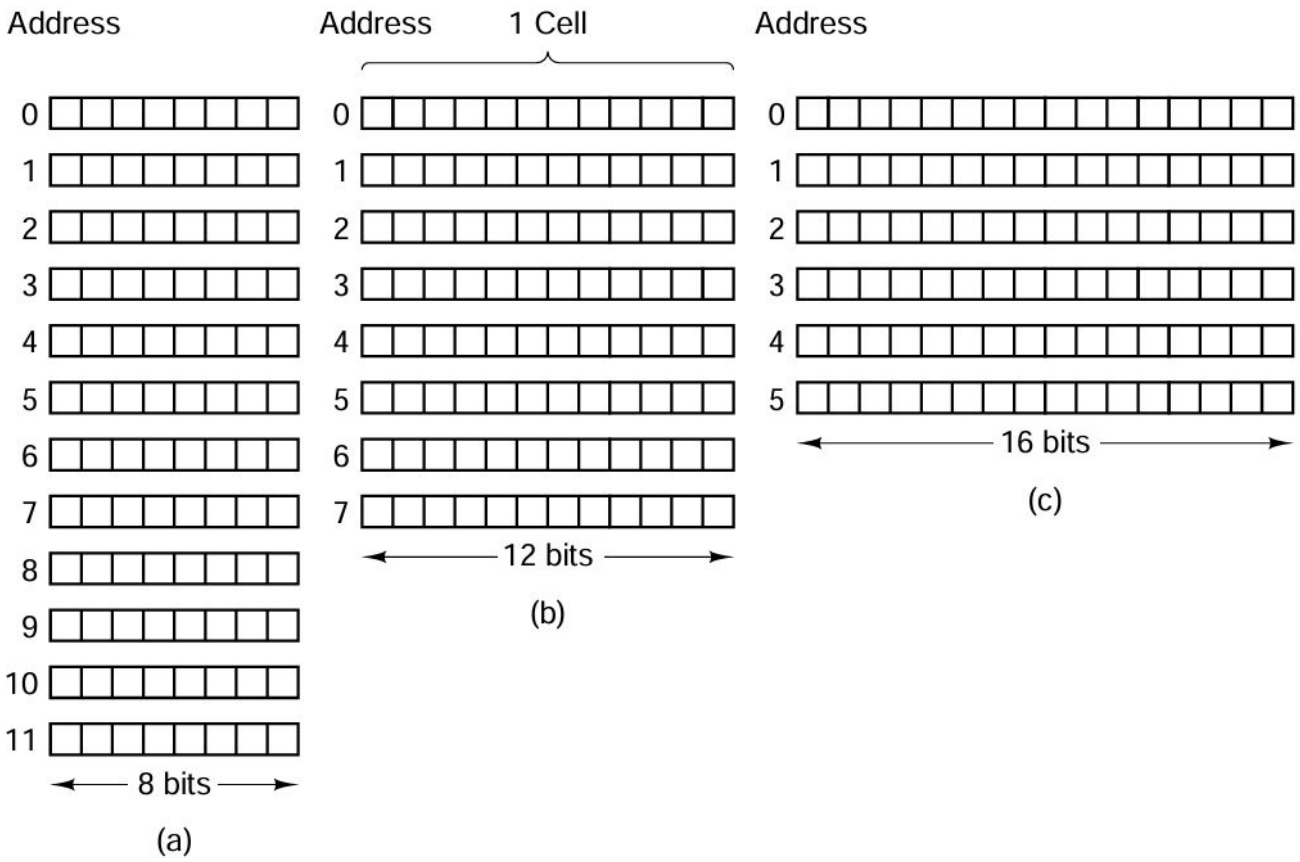


Figure 2-9. Three ways of organizing a 96-bit memory.

Memoria

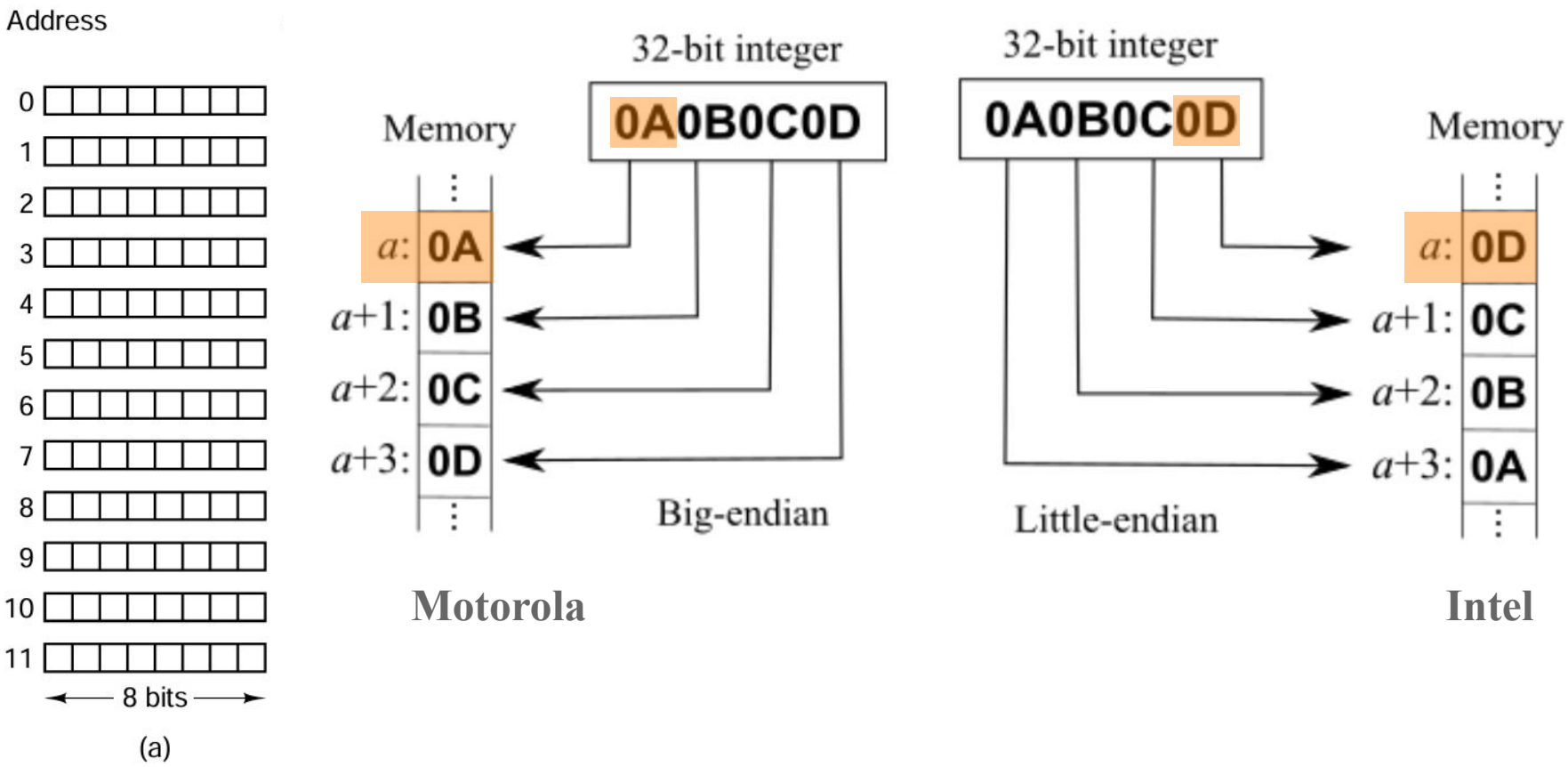
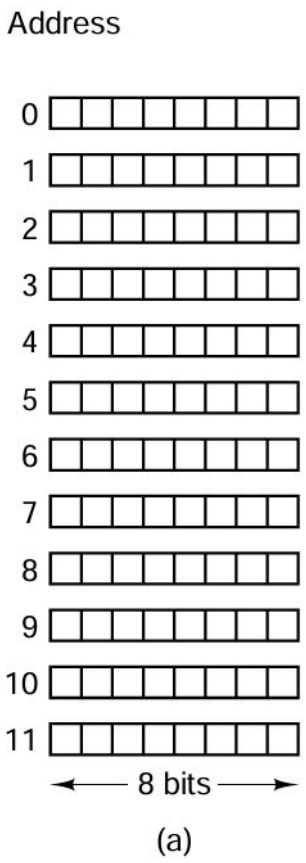
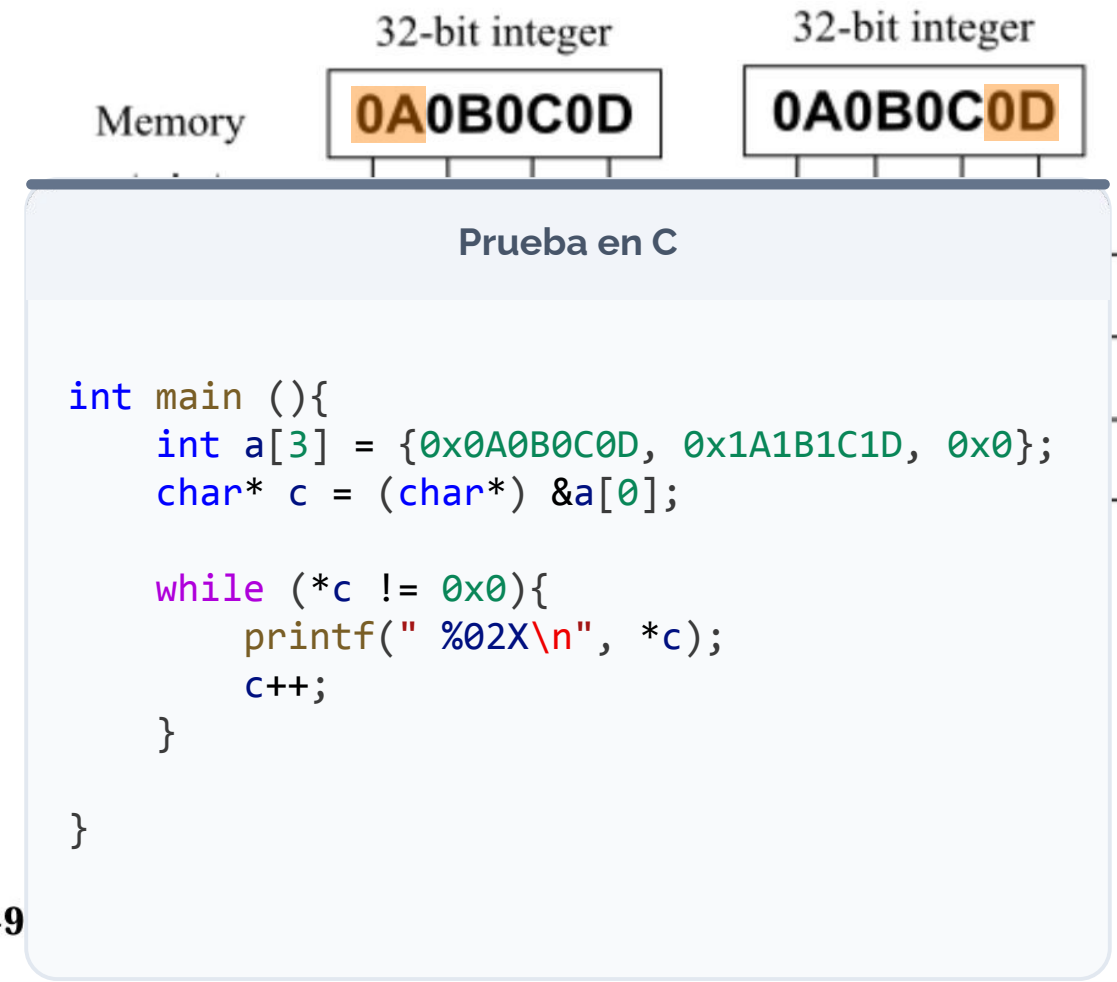


Figure 2-9. Three ways of organizing a 96-bit memory.

Memoria





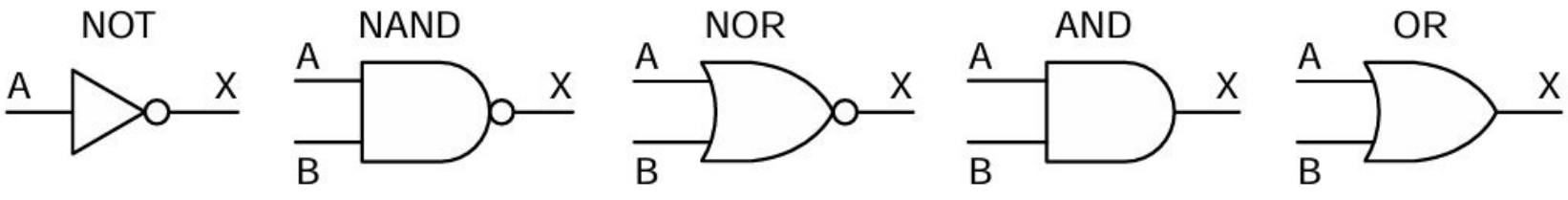
Prueba en C

```c
int main (){
    int a[3] = {0x0A0B0C0D, 0x1A1B1C1D, 0x0};
    char* c = (char*) &a[0];

    while (*c != 0x0){
        printf(" %02X\n", *c);
        c++;
    }

}
```

Figure 2-9

Lógica Digital

Compuertas lógicas



A	X
0	1
1	0

(a)

A	B	X
0	0	1
0	1	1
1	0	1
1	1	0

(b)

A	B	X
0	0	1
0	1	0
1	0	0
1	1	0

(c)

A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

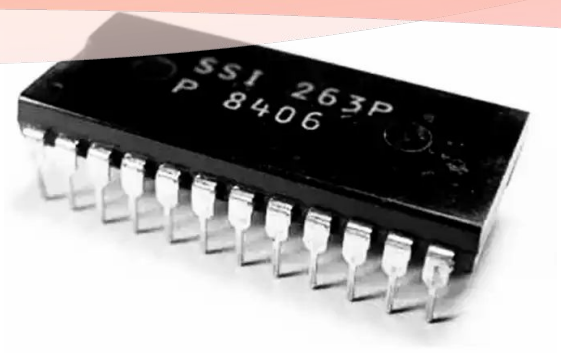
(d)

A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

(e)

Figure 3-2. The symbols and functional behavior for the five basic gates.

Chip



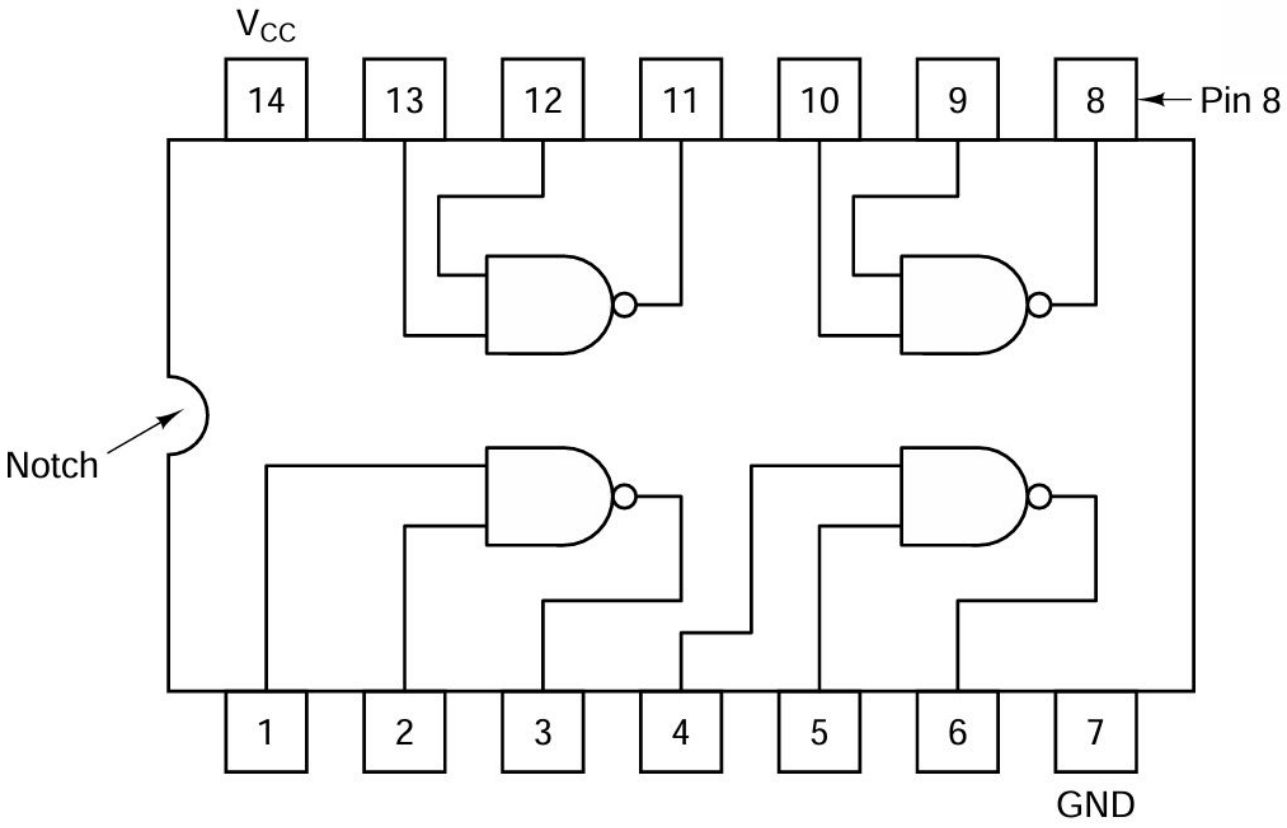
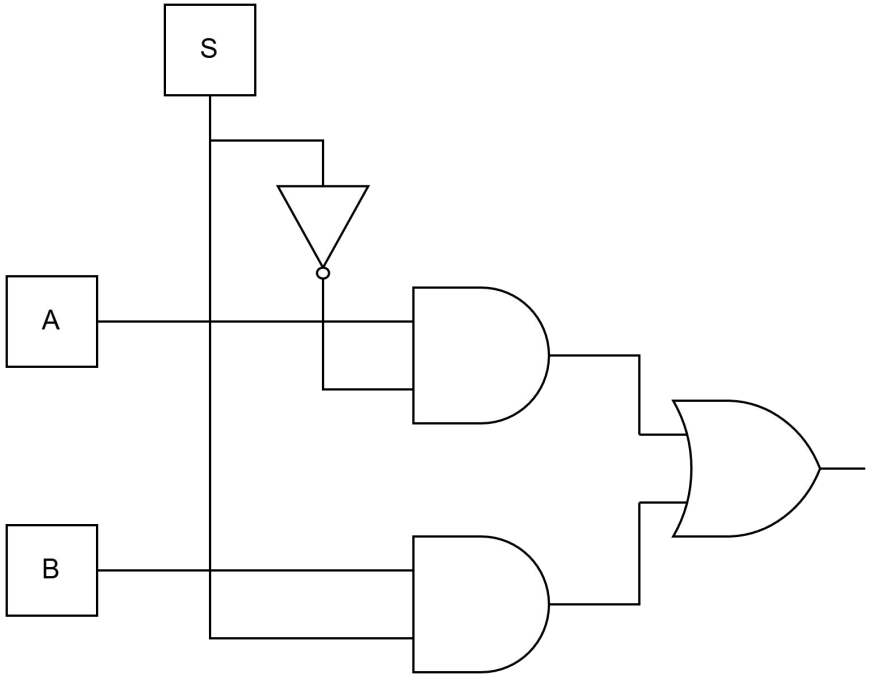
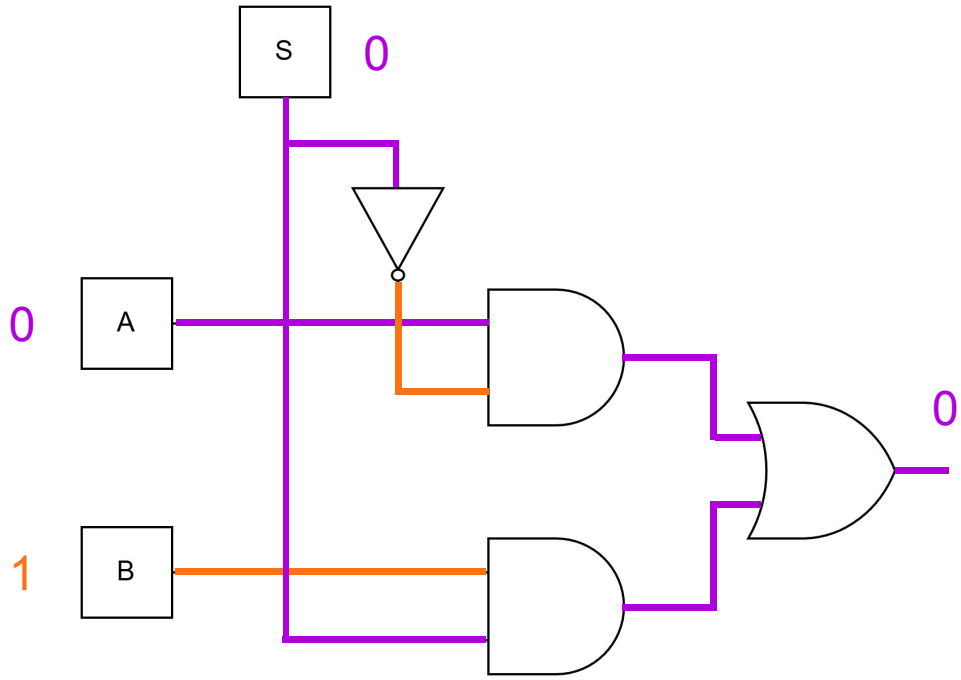


Figure 3-10. An SSI chip containing four gates.

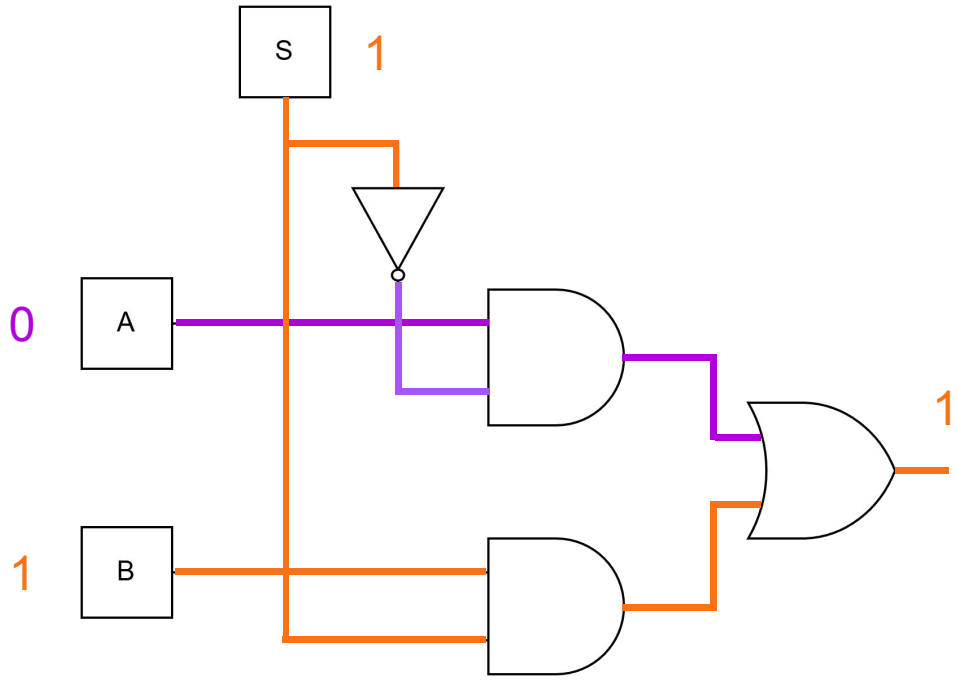
Multiplexor (1 bit)



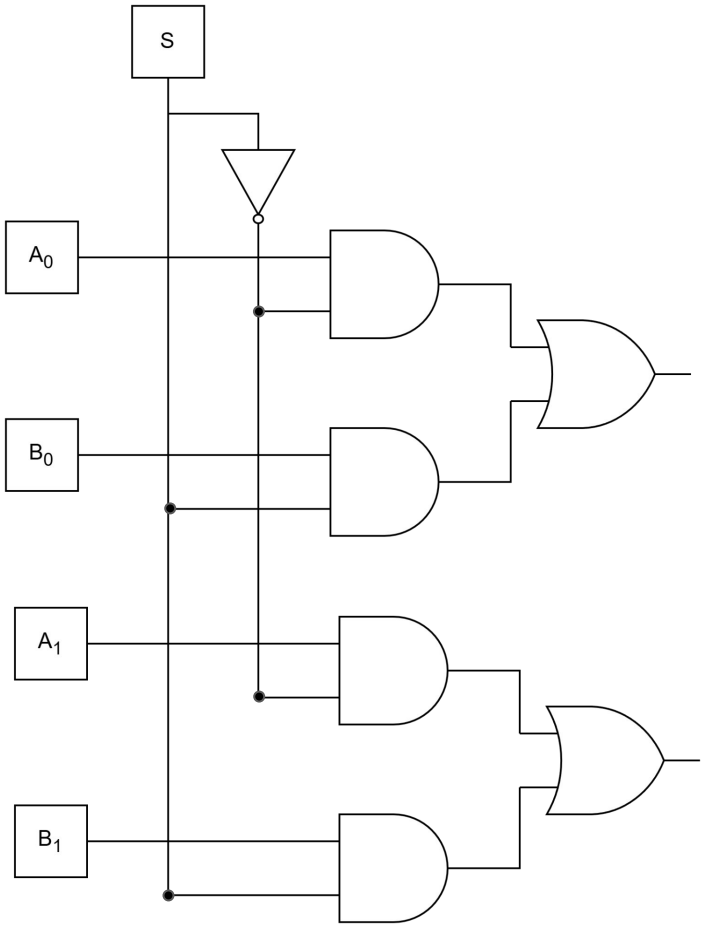
Multiplexor (1 bit)

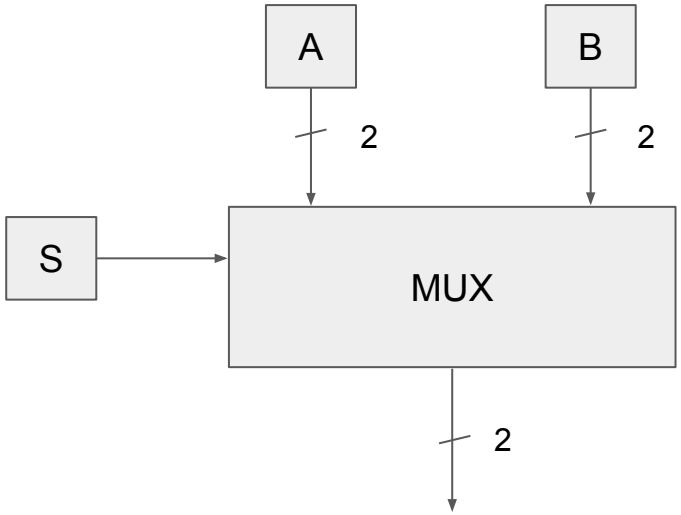


Multiplexor (1 bit)



Multiplexor (2 bits)





Decodificador (3 bits)

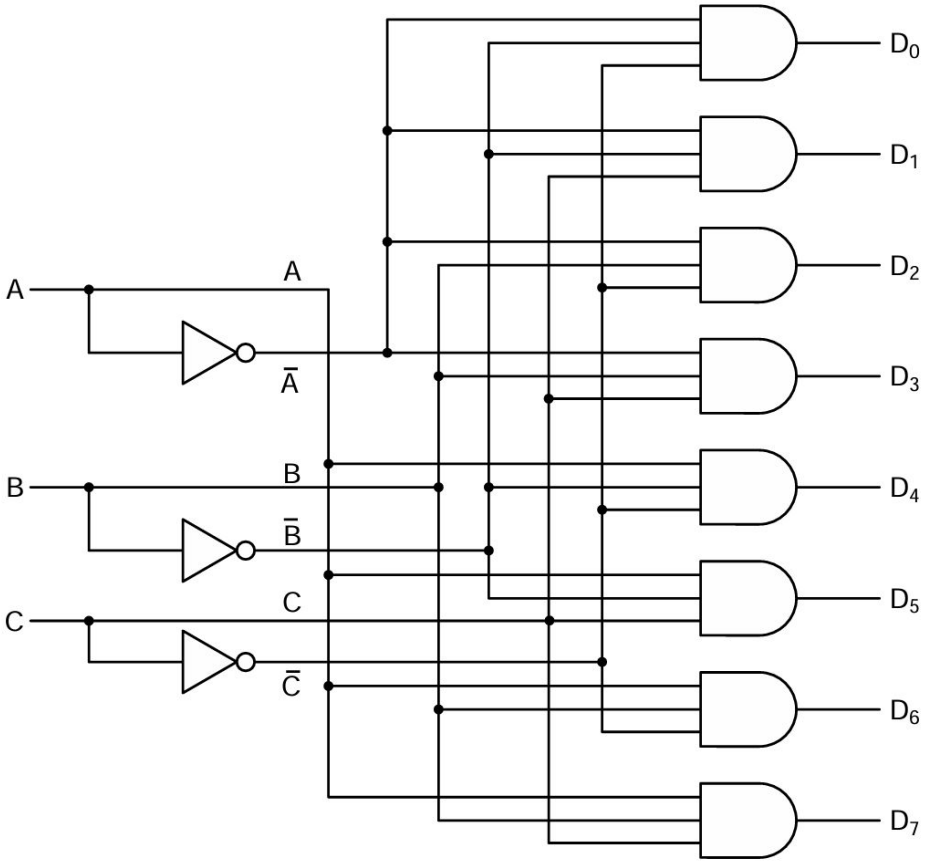
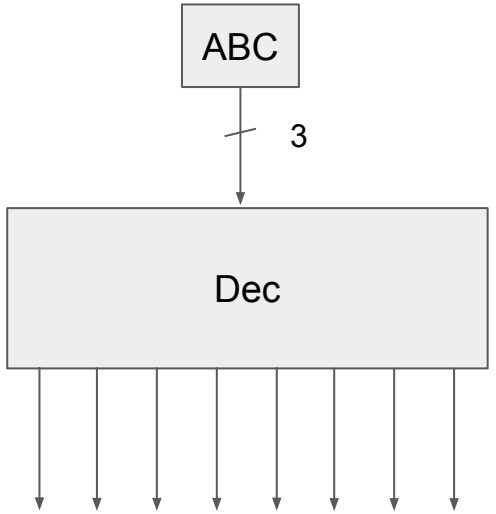


Figure 3-13. A 3-to-8 decoder circuit.



Sumador (1 bit)

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

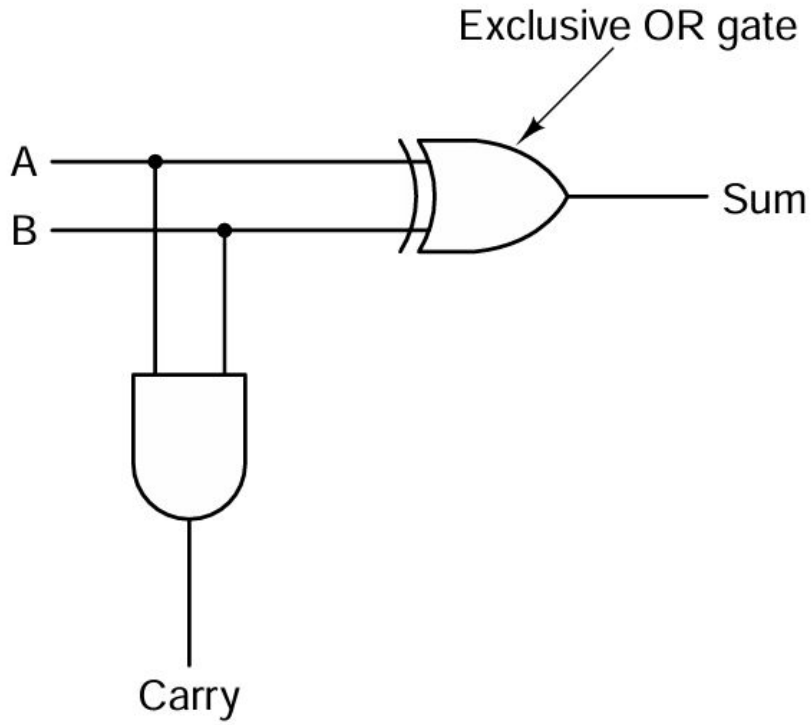
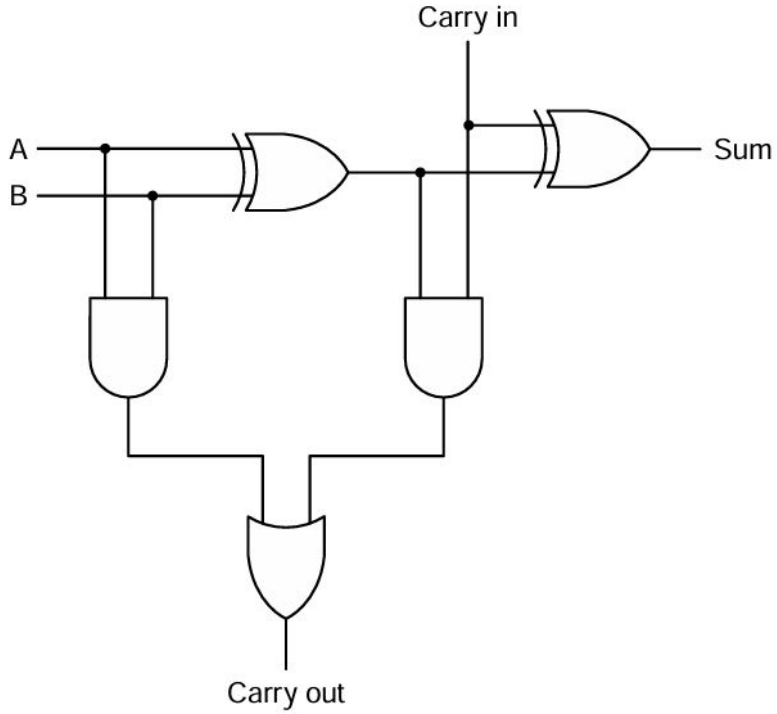


Figure 3-17. (a) Truth table for 1-bit addition. (b) A circuit for a half adder.

Sumador (1 bit con carry in)

A	B	Carry in	Sum	Carry out
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

(a)



(b)

Figure 3-18. (a) Truth table for full adder. (b) Circuit for a full adder.

Unidad Aritmético Lógica (ALU)

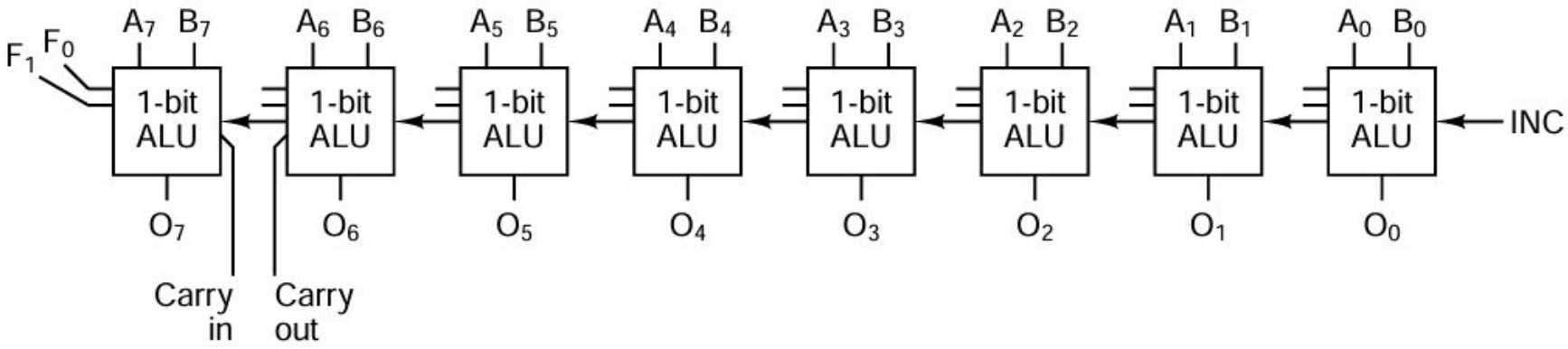
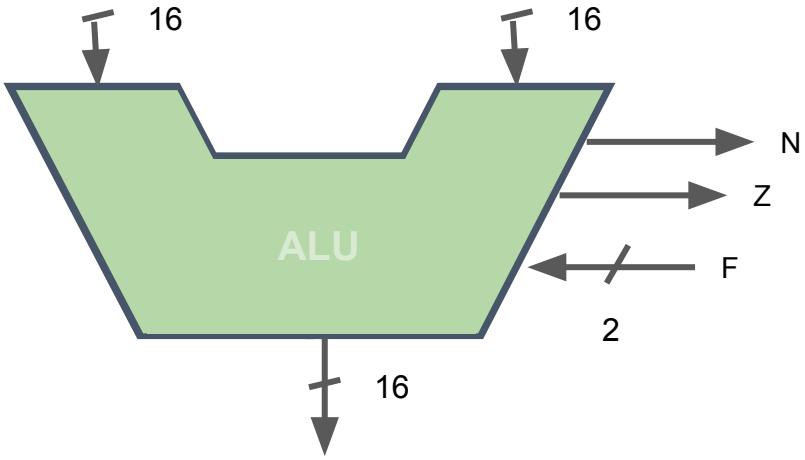


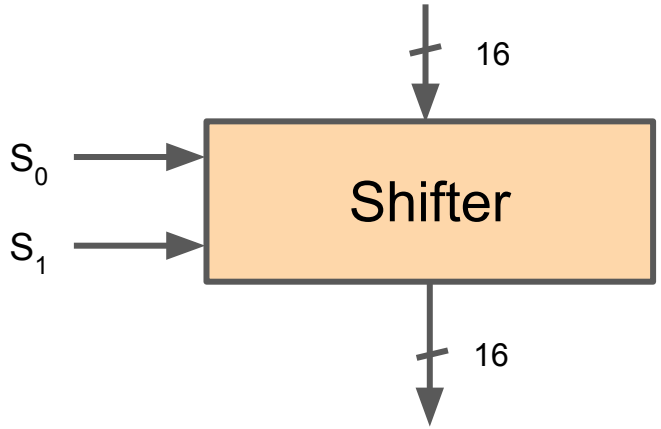
Figure 3-20. Eight 1-bit ALU slices connected to make an 8-bit ALU. The enables and invert signals are not shown for simplicity.

Unidad Aritmético Lógica (ALU)



00 = A + B
01 = A & B
10 = Copia A
11 = Complemento A

Shifter



00 = No Shift
01 = Shift Right 1 bit
10 = Shift Left 1 bit
11 = (sin uso)

Clock

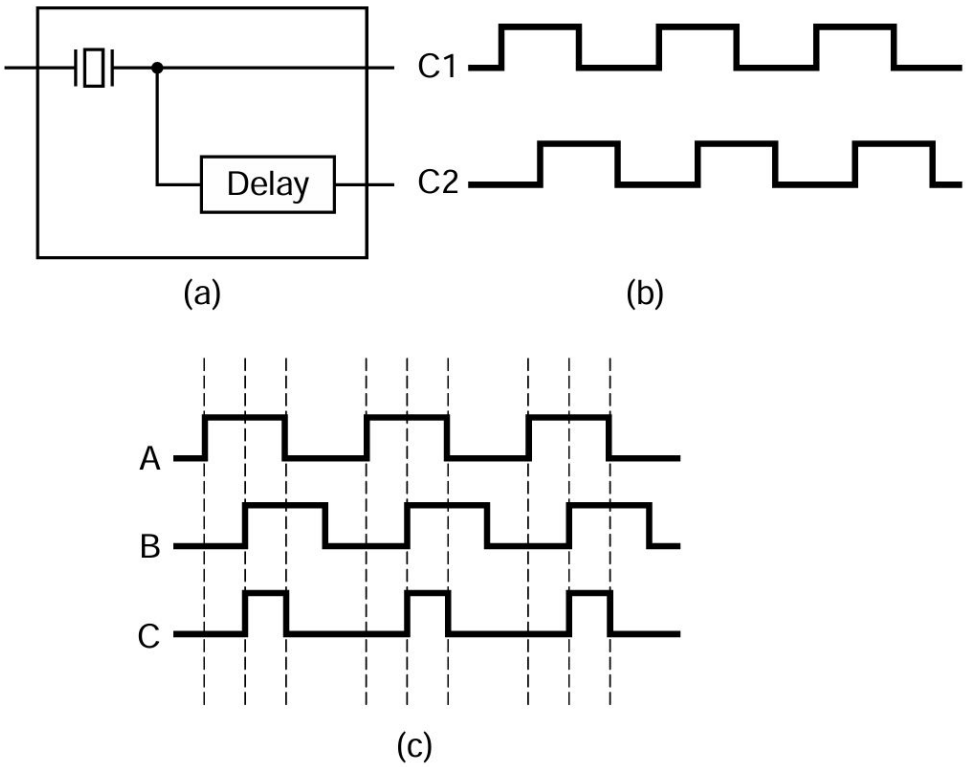
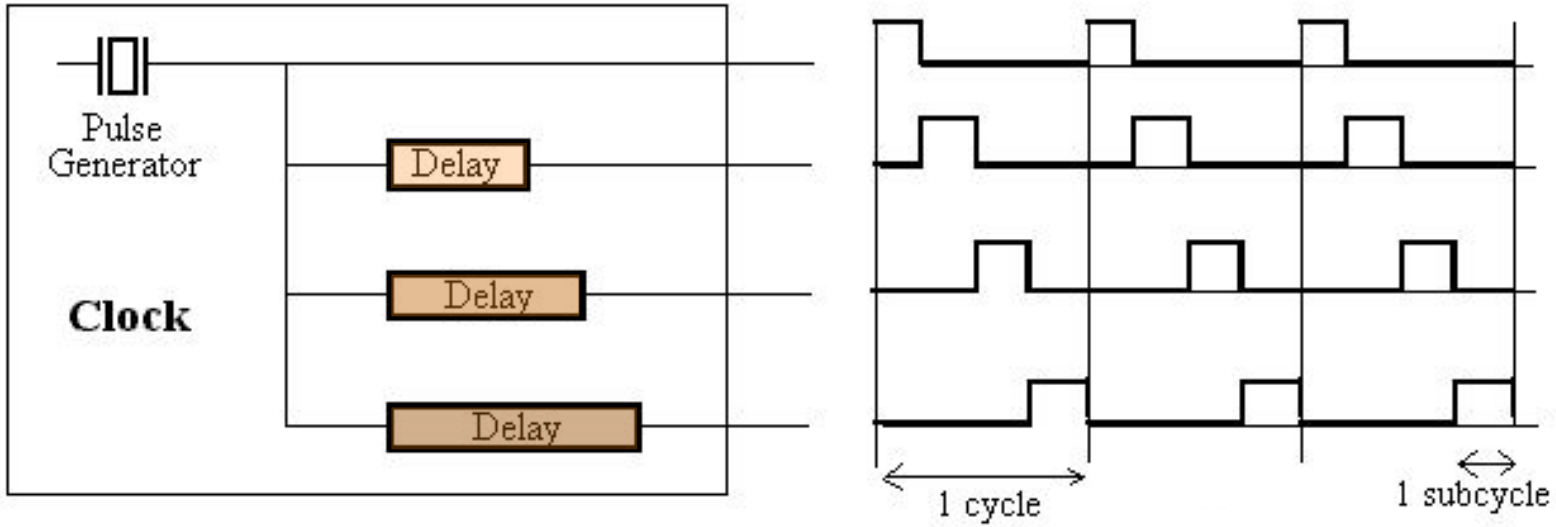


Figure 3-21. (a) A clock. (b) The timing diagram for the clock. (c) Generation of an asymmetric clock.

Clock



Flip Flops

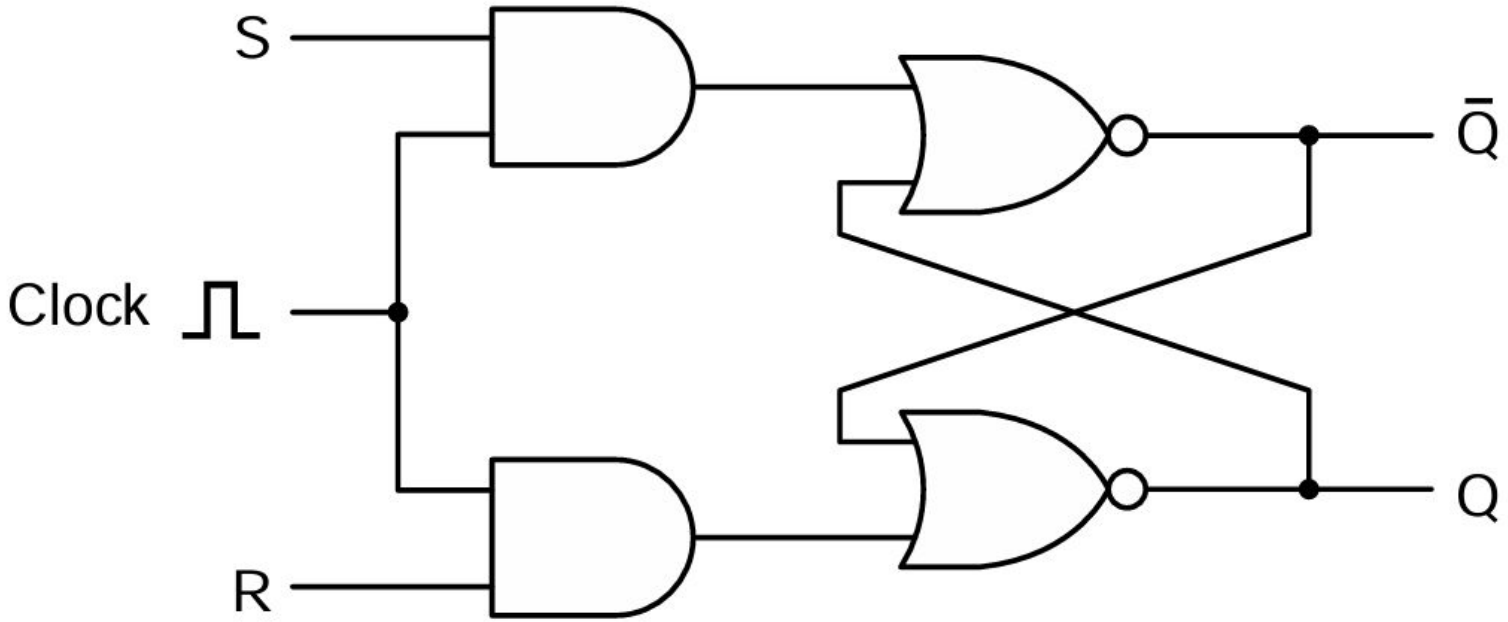


Figure 3-23. A clocked SR latch.

Flip Flops

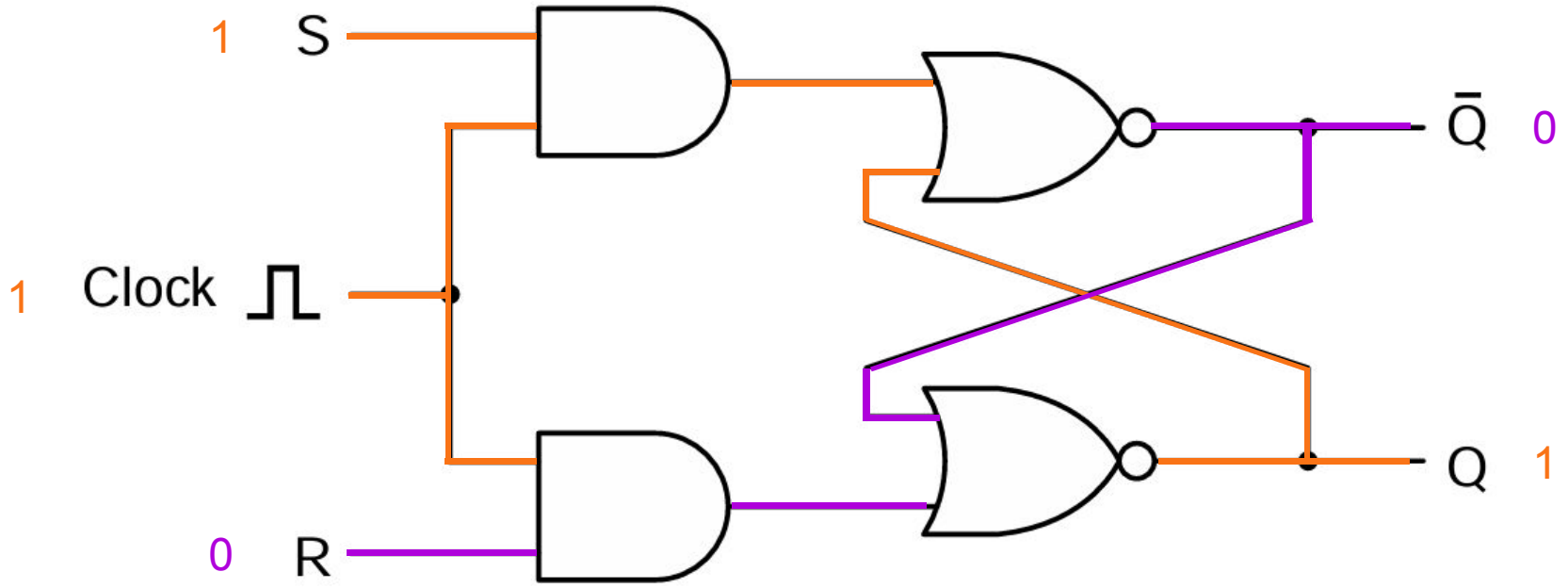


Figure 3-23. A clocked SR latch.

Flip Flops

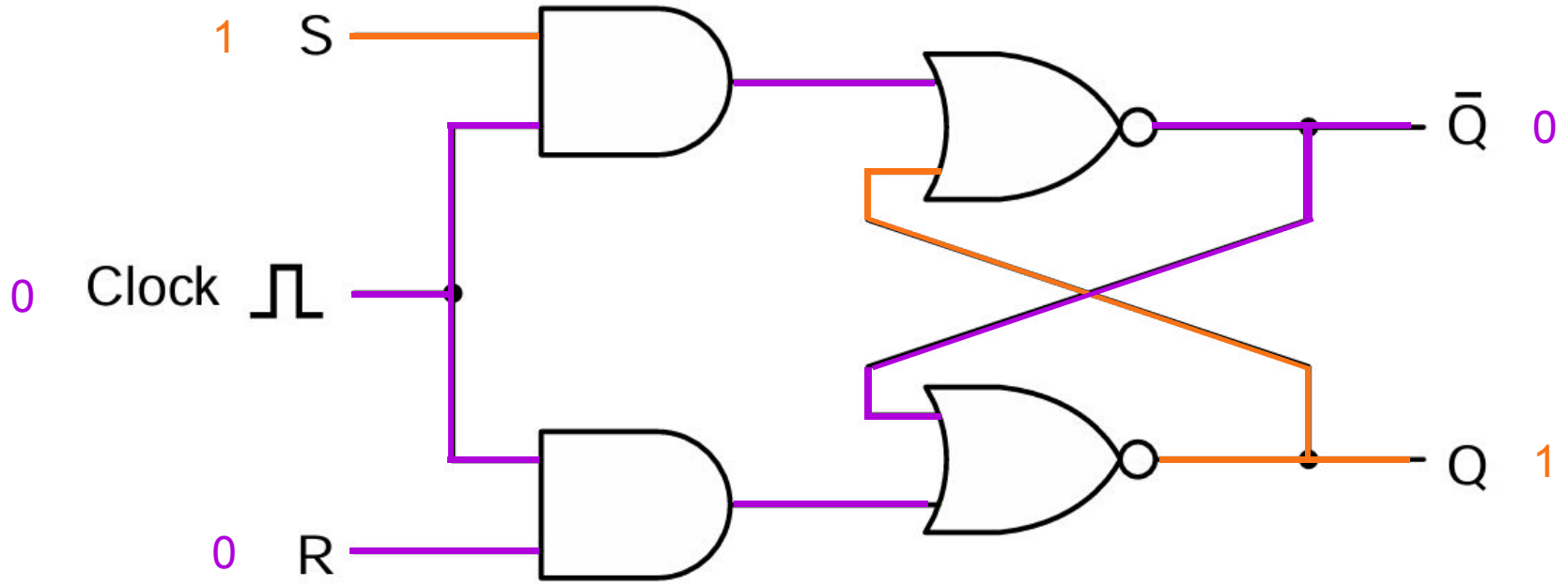


Figure 3-23. A clocked SR latch.

Flip Flops

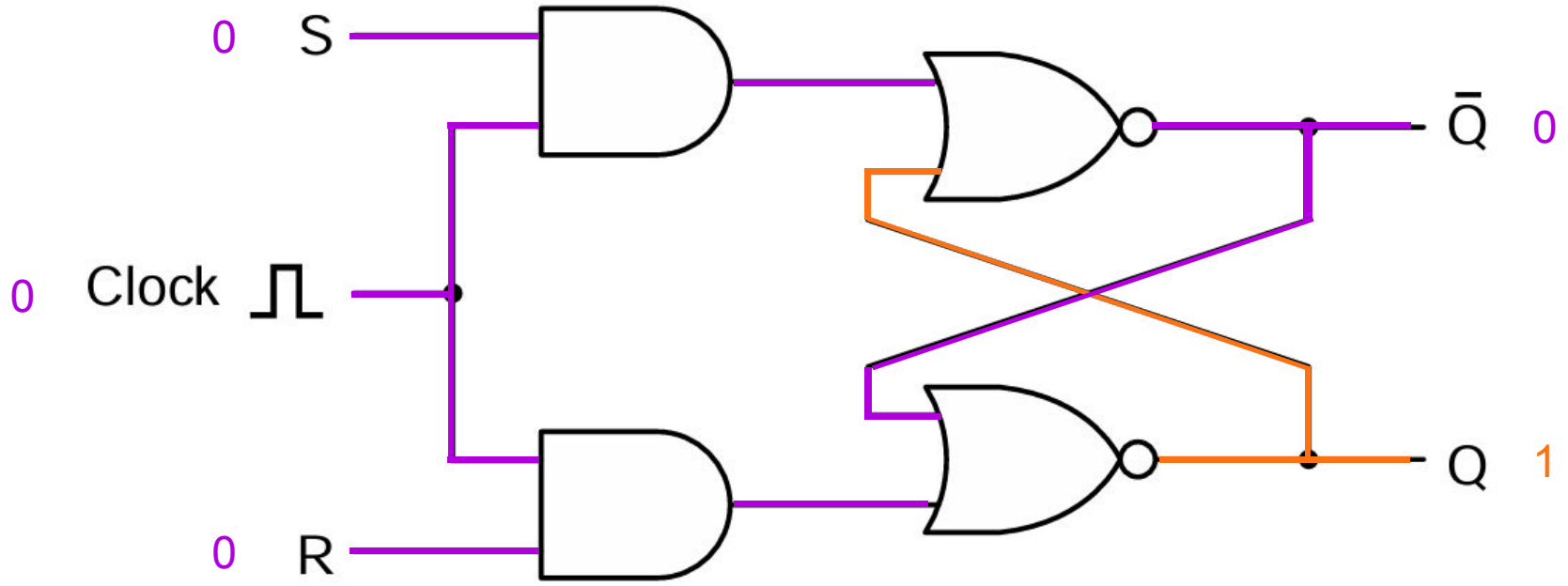


Figure 3-23. A clocked SR latch.

Flip Flops

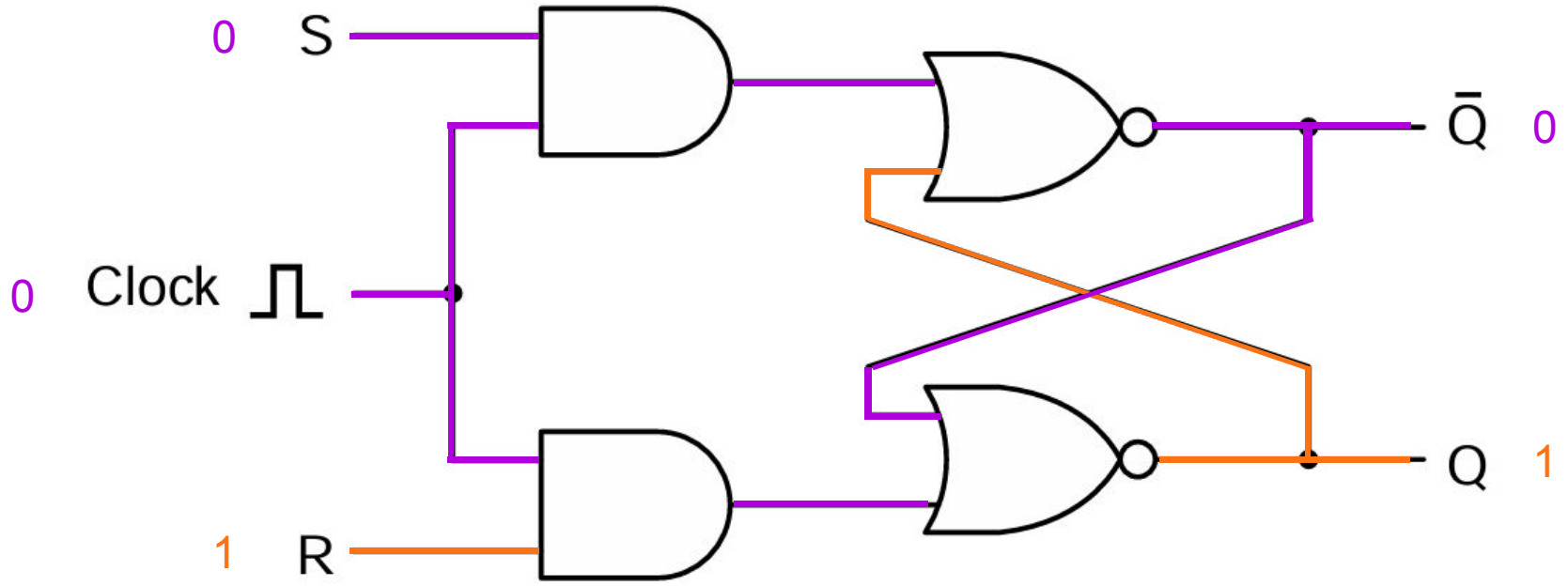


Figure 3-23. A clocked SR latch.

Flip Flops

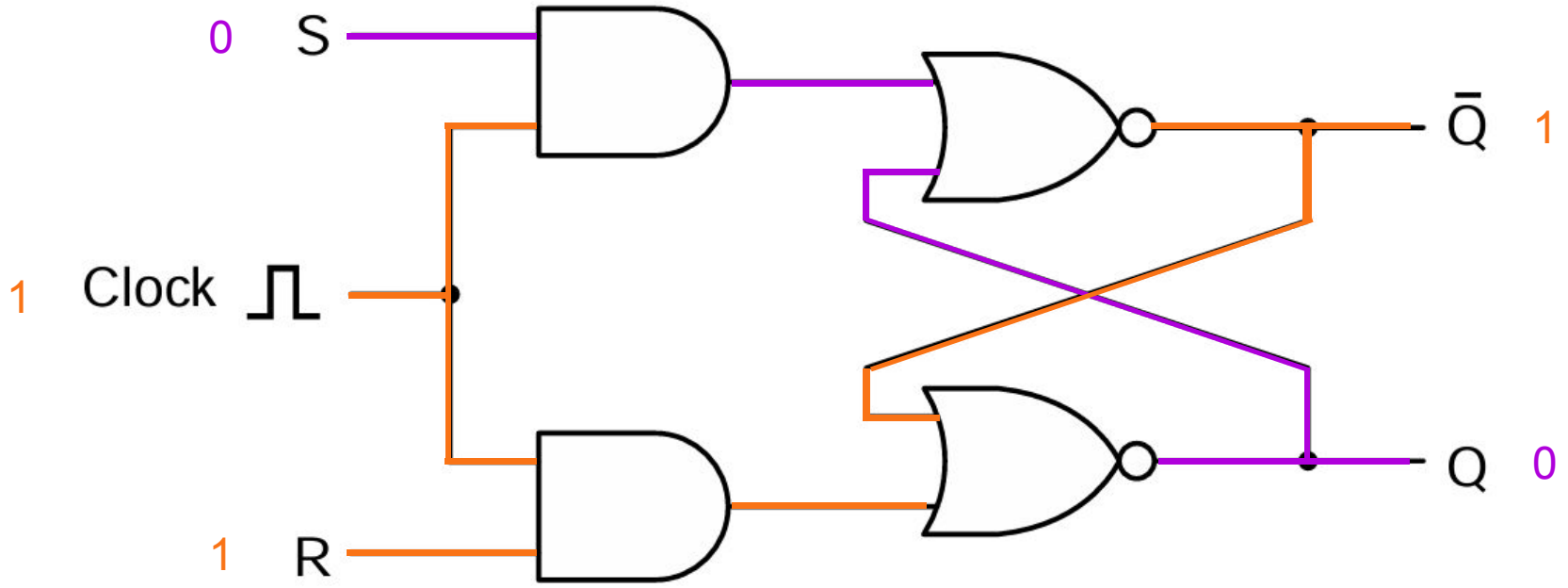


Figure 3-23. A clocked SR latch.

Flip Flops

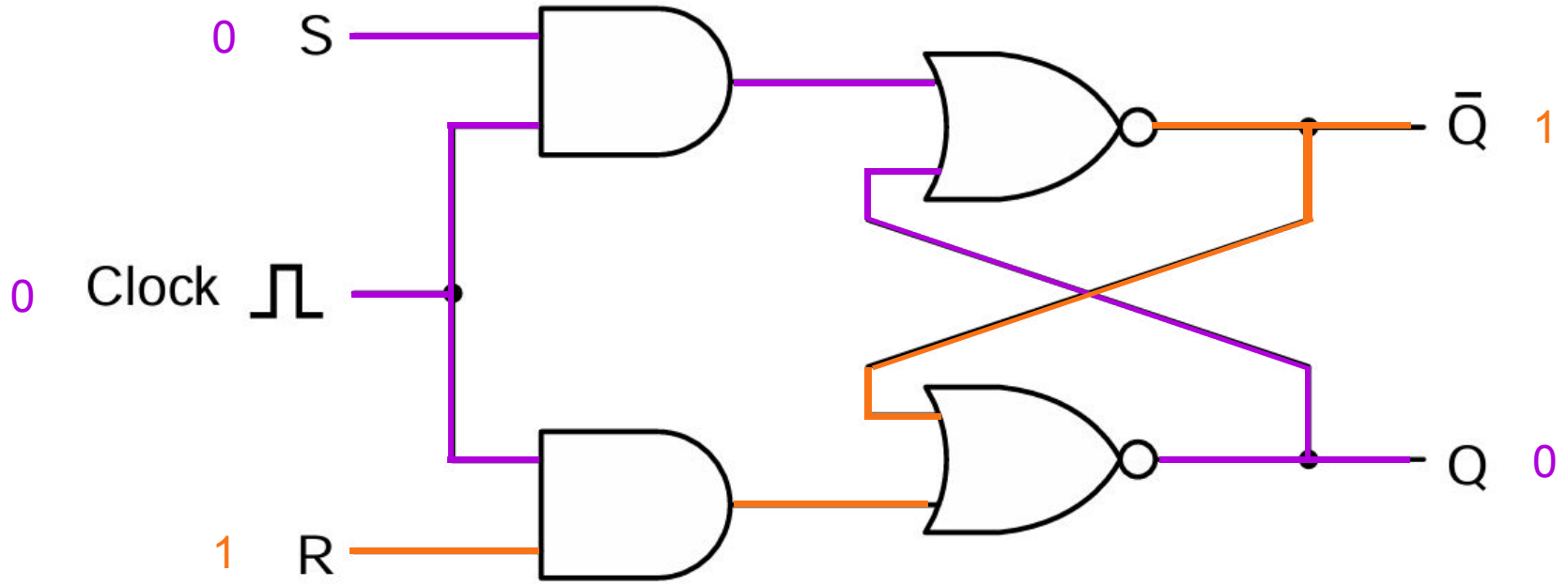


Figure 3-23. A clocked SR latch.

Flip Flops

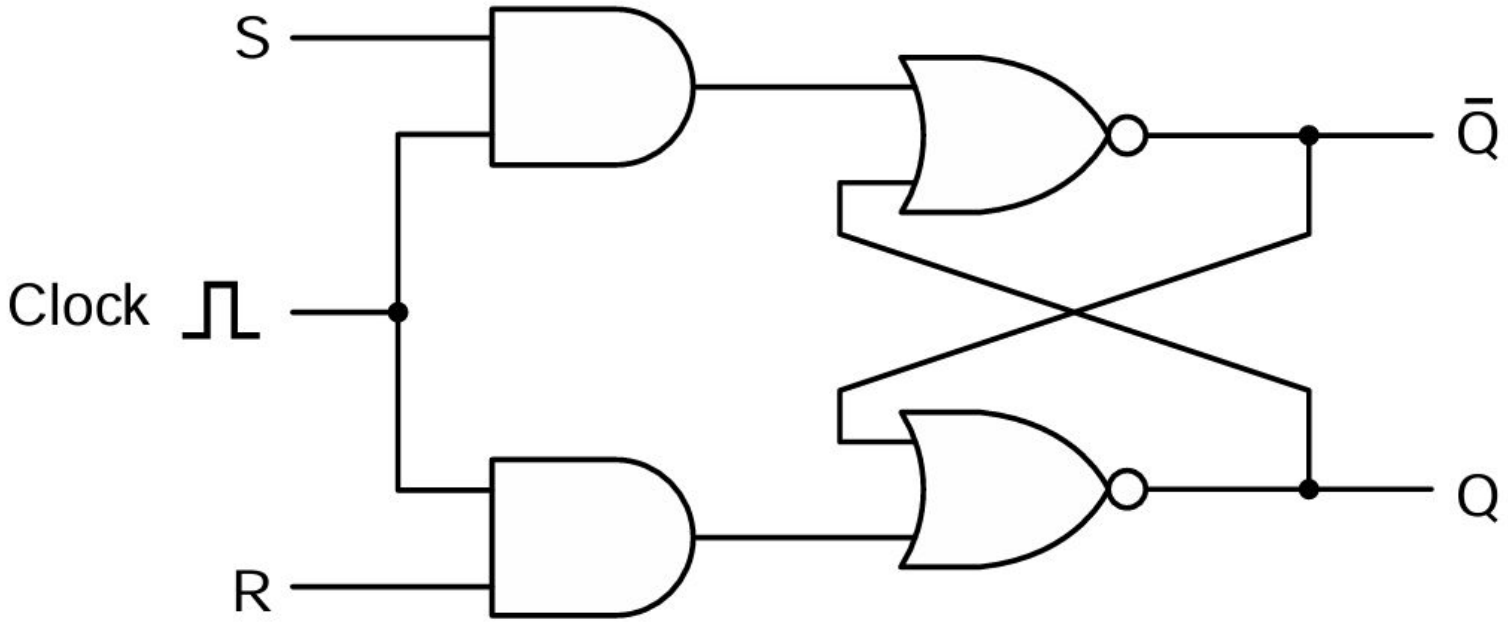


Figure 3-23. A clocked SR latch.

Registro

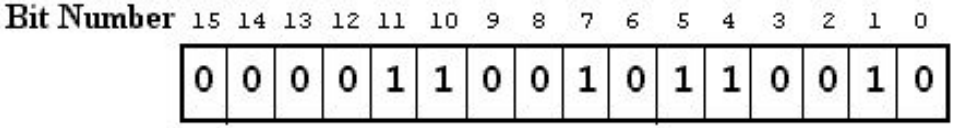


Fig 4-1. A 16-bit register can hold 16 bits of information

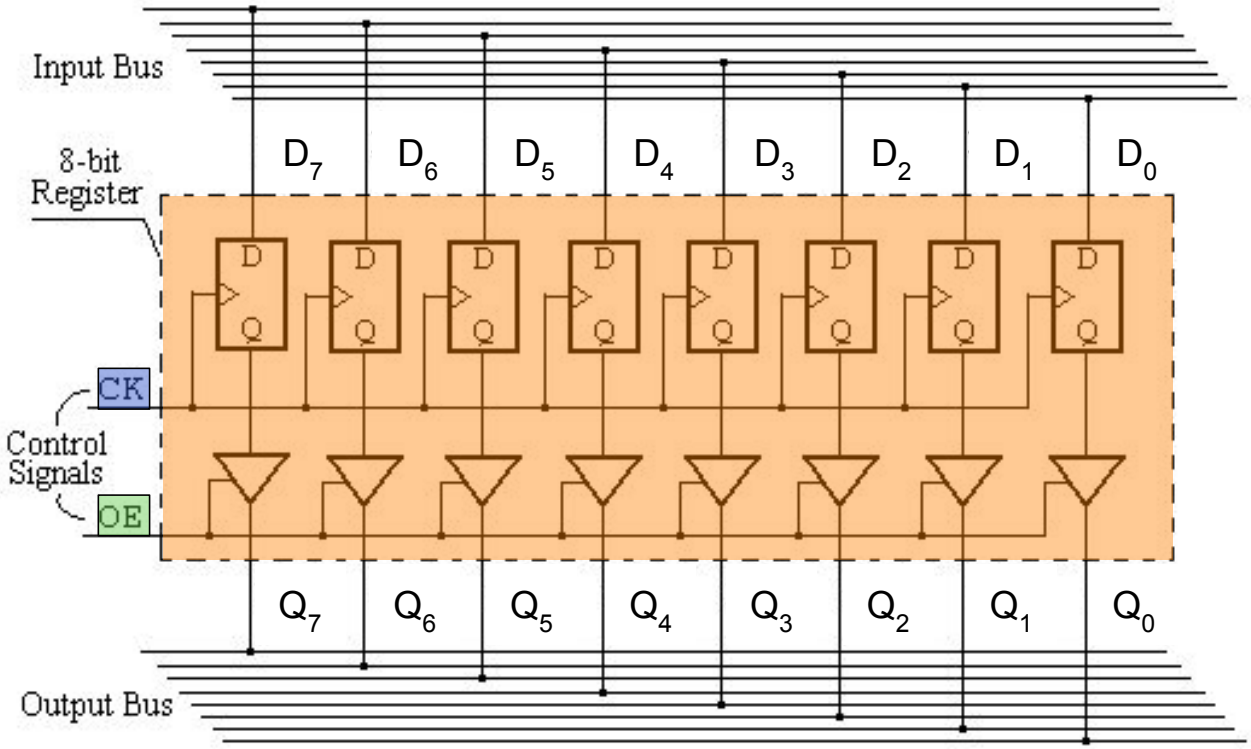


Fig 4-2 (a) Detail of an 8-bit register connected to an input bus and an output bus

Registro

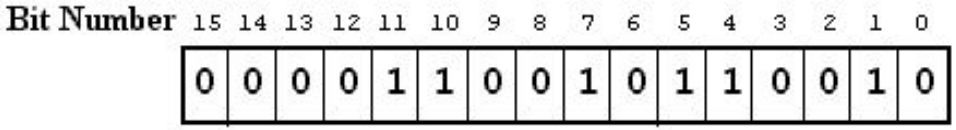


Fig 4-1. A 16-bit register can hold 16 bits of information

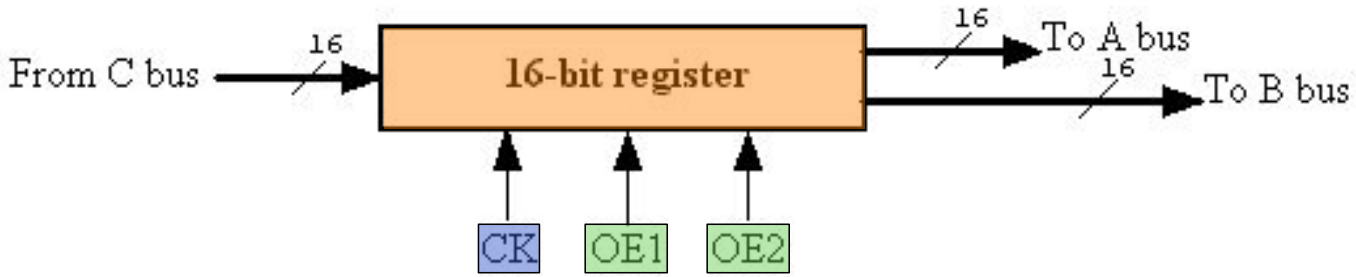
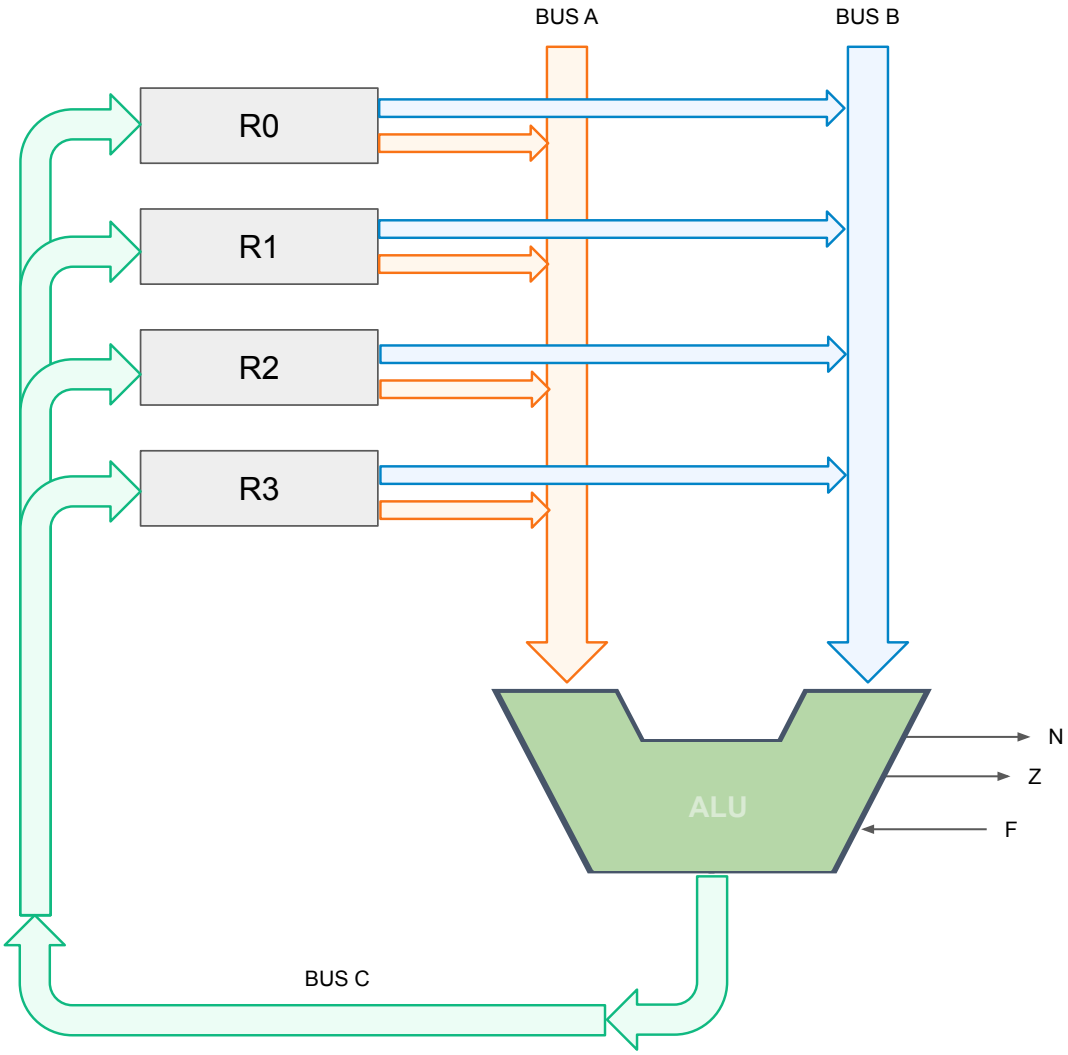


Fig 4-2 (b) Symbolic representation of a 16-bit register with one input bus and two output buses

Uniendo algunas piezas



Máquina multinivel

Máquina Multinivel

y Lógica Digital

Fundamentos de

Arquitectura de Computadoras